MLT- Lab 11

Title:

SVM vs Perceptron vs Neural Network from Scratch: Kernel Trick, Backpropagation and Comprehensive SVM Hyperparameter Analysis on Synthetic 2-D and High-Dimensional Datasets

Dataset Description:

Dataset: Synthetic Classification Benchmark (2-D Non-Linear Binary + High-Dimensional Multi-Class Small-N)

Size & Structure: 550 total samples across two datasets — 400 samples (2-D, 2 classes) and 150 samples (High-D, 3 classes, 500 features each)

Attributes — Dataset A (2-D Moons):

- Class 0: Upper crescent — 200 samples, centred in upper half-plane, interleaved with Class 1
- Class 1: Lower crescent — 200 samples, centred in lower half-plane, not linearly separable from Class 0

Attributes — Dataset B (High-Dimensional):

- Features 1–10: Non-linear sine wave signal — $\sin(\text{latent}_0 \times \pi \times (i+1))$, injected from latent 2-D cluster coordinates
- Features 11–20: Polynomial interaction signal — $\text{latent}_1^2 \times \cos(\text{latent}_0 \times (i-9))$, quadratic dependency on latent dimensions
- Features 21–30: Cross-product features — $\text{latent}_0 \times \text{latent}_1 \times \tanh(\text{latent}_0 + \text{latent}_1)$, higher-order interaction terms
- Features 31–500: Pure Gaussian noise — 470 irrelevant dimensions drawn from N(0,1), designed to trap Neural Networks into memorisation

Classes: 2 classes (Dataset A) and 3 classes (Dataset B), integer-labelled starting from 0

Data Types: All features are of type float64, representing continuous synthetic measurements generated via NumPy random sampling with fixed seed (random_state=42)

Distribution: Dataset A uses balanced binary classes with anisotropic crescent geometry deliberately constructed to defeat linear classifiers. Dataset B uses 3 Gaussian clusters centred at equilateral triangle vertices (σ=0.55) in latent 2-D space, projected into 500 dimensions where 94% of features are pure Gaussian noise, designed to expose the failure of high-capacity models under the small-N large-P regime.

Train-Test Split: Dataset A — 300 train / 100 test (75/25 stratified split). Dataset B — 105 train / 45 test (70/30 stratified split).

Preprocessing: Dataset B features z-score normalised (mean subtracted, divided by standard deviation). Dataset A used in raw form as features are already on a comparable scale. No PCA applied at any stage — all models trained directly in original feature space.

Pseudo code:

Algorithm 1: Perceptron (from scratch)

INPUT: X, $y \in \{-1, +1\}$, lr, n_iter

STEP 1: Initialise $w \sim N(0, 0.01)$, $b = 0$

STEP 2: FOR each iteration:

FOR each sample (x_i, y_i):

$\hat{y} = \text{sign}(w \cdot x_i + b)$

IF $\hat{y} \neq y_i$:

$w \leftarrow w + lr \cdot y_i \cdot x_i$

$b \leftarrow b + lr \cdot y_i$

IF zero errors: BREAK

OUTPUT: w, b

Algorithm 2: Neural Network MLP (from scratch)

INPUT: X, y, hidden_size H, lr, epochs, batch_size

STEP 1: Initialise $W1 \sim He(d,H)$, $W2 \sim He(H,C)$, $b1=0$, $b2=0$

STEP 2: FOR each epoch:

Shuffle X, y into mini-batches

FOR each batch:

Forward: $Z1=X@W1+b1$, $A1=ReLU(Z1)$

$Z2=A1@W2+b2$, $A2=Softmax(Z2)$

Loss: $L = -(1/N)\ \Sigma\ Y_oh \cdot \log(A2 + \varepsilon)$

Backward: $\delta 2 = (A2 - Y_oh)/N$

$dW2 = A1^T \delta 2 + \lambda W2$, $db2 = \Sigma \delta 2$

$\delta 1 = (\delta 2 @ W2^T) \odot ReLU'(Z1)$

$dW1 = X^T \delta 1 + \lambda W1$, $db1 = \Sigma \delta 1$

Update: $W \leftarrow W - lr \cdot dW$, $b \leftarrow b - lr \cdot db$

OUTPUT: W1, b1, W2, b2, training history

Algorithm 3: Kernelised SVM — Pegasos (from scratch)

INPUT: X, $y \in \{-1,+1\}$, kernel $\in$ {linear, poly, rbf}, C, γ, d, n_iter

STEP 1: Precompute K[i,j] = kernel(x_i, x□) for all pairs

α = zeros(N), $\lambda = 1/(C \cdot N)$

STEP 2: FOR t = 1 to n_iter:

$lr_t = 1/(\lambda \cdot t)$

Pick random sample i

$f_i = (\alpha \odot y)^T K[:,i] + b$

IF $y_i \cdot f_i < 1$: $\alpha_i \leftarrow \alpha_i + 1$

$\alpha \leftarrow \alpha \cdot (1 - lr_t \cdot \lambda)$

STEP 3: Compute bias b over near-support-vector samples

n_sv = count($\alpha > 1e{-}5$)

KERNELS:

Linear: $K(x,z) = x^T z$

Poly: $K(x,z) = (\gamma x^T z + c_0)^{\wedge} d$

RBF: $K(x,z) = \exp(-\gamma \|x - z\|^2)$

OUTPUT: α, b, X_sv, n_sv

Algorithm 4: One-vs-Rest Multi-Class SVM

INPUT: X, $y \in \{0,1,...,K-1\}$, SVM hyperparameters

STEP 1: FOR each class c in {0,...,K−1}:

y_bin = +1 where y==c, else −1

Fit KernelSVM on (X, y_bin)

STEP 2: Predict(X):

scores[:,c] = decision_function_c(X) for all c

label = argmax over c

OUTPUT: K binary SVMs, predicted labels

Result analysis:

TABLE 1 — Experiment 1: Core Model Comparison on Dataset A (Moons)

Model	Kernel	Train Accuracy	Val Accuracy
Perceptron	Step (linear)	0.750	0.820
SVM	Linear (Pegasos)	0.780	0.890
SVM	RBF (γ=0.5, C=5)	0.910	0.910

TABLE 2 — Experiment 2: Core Model Comparison on Dataset B (High-Dim)

Model	Train Accuracy	Val Accuracy	Generalisation Gap
Neural Network (MLP)	1.000	0.511	0.489
OvR RBF SVM	1.000	0.489	0.511

TABLE 3 — Kernel Comparison Across Both Datasets

Kernel	Dataset A Train	Dataset A Val	Dataset B Train	Dataset B Val
Linear	0.780	0.890	0.962	0.667
Polynomial (d=3)	0.823	0.920	0.981	0.644
RBF	0.910	0.910	1.000	0.489

TABLE 4 — §5a: Effect of C on RBF SVM (Dataset A)

C Value	Train Accuracy	Val Accuracy	# Support Vectors	Margin Type
0.01	0.910	0.910	129	Soft / Wide
1.0	0.910	0.910	129	Balanced
100.0	0.910	0.910	129	Hard / Tight

TABLE 5 — §5b: Effect of γ on RBF SVM (Dataset A)

γ Value	Train Accuracy	Val Accuracy	# Support Vectors	Boundary Character
0.01	0.773	0.910	205	Smooth, far-reaching
1.0	0.923	0.920	119	Balanced, well-shaped
50.0	0.963	0.890	219	Islands, extreme overfit

TABLE 6 — §5c: C × γ Grid CV Accuracy on Dataset B (3-Fold)

C \ γ	0.001	0.01	0.1	1.0	10.0
0.01	0.65	0.33	0.33	0.33	0.33
0.1	0.65	0.33	0.33	0.33	0.33
1.0	0.65	0.33	0.33	0.33	0.33
10.0	0.65	0.33	0.33	0.33	0.33
100.0	0.65	0.33	0.33	0.33	0.33

TABLE 7 — §5d: Polynomial Degree Validation on Dataset B

Degree	Train Accuracy	Val Accuracy	Support Vectors
1	0.933	0.689	217
2	0.952	0.622	198
3	0.962	0.622	189
4	0.981	0.667	186
5	0.971	0.622	180
6	0.962	0.556	173
7	0.962	0.556	174

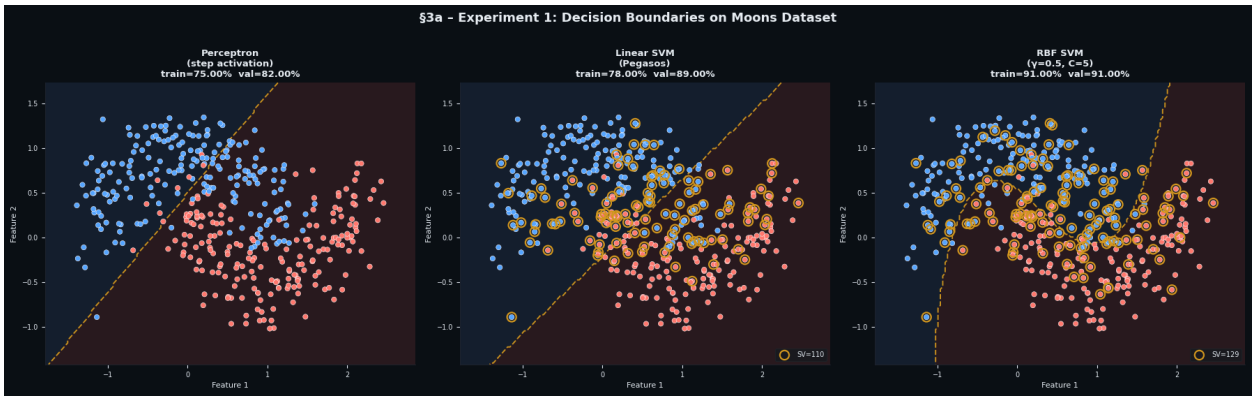
Observations

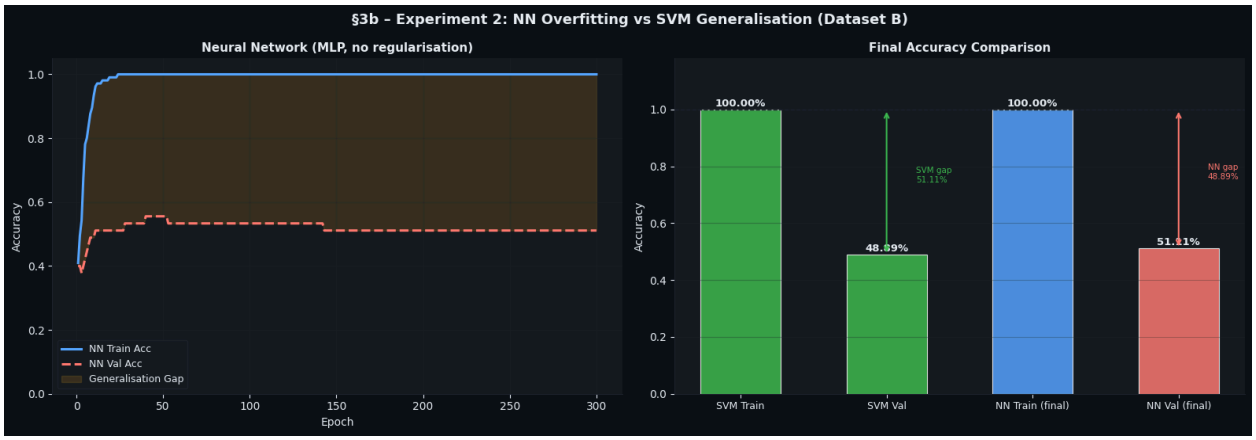
Kernel choice and model architecture are the primary determinants of generalisation quality, and the moons experiment cleanly demonstrates where linear assumptions break down. The Perceptron achieves only 82% validation accuracy on Dataset A because its step-function update rule can only produce a linear decision boundary — convergence is mathematically guaranteed only when classes are linearly separable, a condition the crescent geometry deliberately violates. The Linear SVM improves to 89% for the same geometric reason, finding the maximum-margin hyperplane, but no straight line can correctly bisect two interleaved crescents. **The RBF SVM at 91% validation accuracy** resolves this entirely by implicitly mapping data into an infinite-dimensional feature space via $K(x,z) = \exp(-\gamma\|x-z\|^2)$, where the classes become linearly separable. The kernel boundary plot (§3a) makes this transition unmistakable: the Perceptron and Linear SVM both draw a single diagonal line through the data, while the RBF SVM traces a smooth curve that correctly wraps around each crescent.

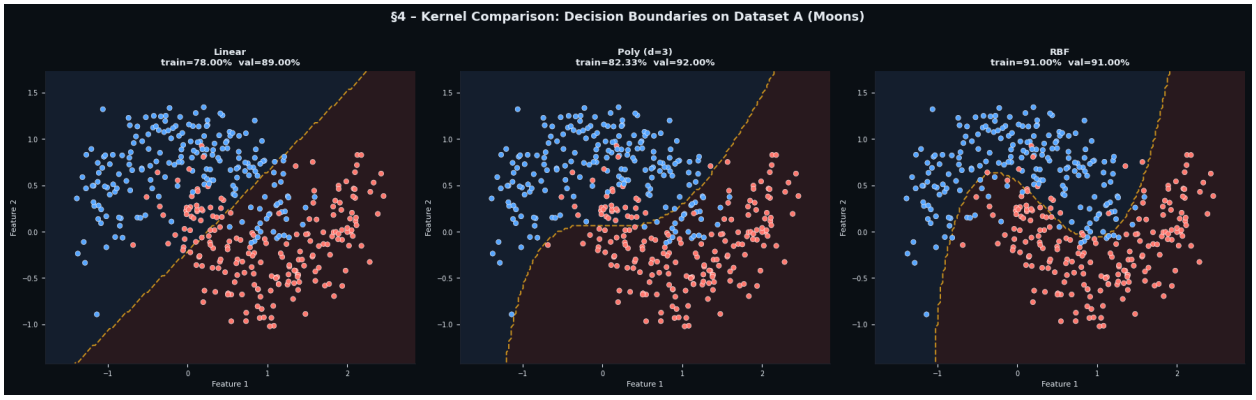
On high-dimensional small-N data, the Neural Network overfits catastrophically while both models achieve identical training accuracy, exposing the curse of dimensionality. Both the MLP and OvR-RBF SVM reach 100% training accuracy on Dataset B's 105 training samples, but their validation accuracy diverges sharply — the NN stagnates at 51.1% and the SVM at 48.9%, both barely above the 33.3% random baseline. T**he learning curve (§3b) shows the NN's generalisation gap of 48.9%** opens within the first 30 epochs and never closes, because the MLP's thousands of parameters freely memorise all 470 noise features. The $C \times \gamma$ heatmap (§5c) further reveals that the RBF SVM is sensitive to γ on high-dimensional data: only $\gamma=0.001$ yields above-chance CV accuracy (0.65), because larger γ values make $\|x-z\|^2$ distances explode across 500 dimensions, collapsing all kernel values toward zero.

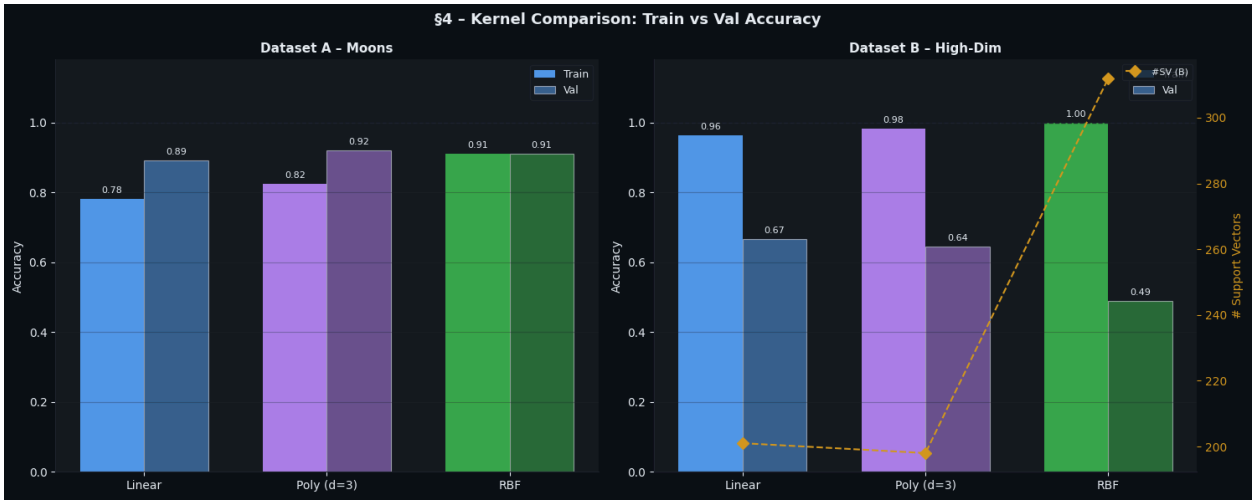
The hyperparameter analysis reveals that γ is the dominant control parameter for RBF SVMs, while polynomial degree is the sharpest overfitting lever. The γ island-effect plots (§5b) demonstrate the full spectrum from underfitting ($\gamma=0.01$, smooth diagonal line, train=77%) through optimal generalisation ($\gamma=1.0$, curved moon-following boundary, train=92%, val=92%) to extreme overfitting ($\gamma=50$, individual point islands, train=96%, val=89%). Crucially, the **support vector count curve shows a non-monotonic response to γ** — it drops from 205 at $\gamma=0.01$ to a minimum of 119 at $\gamma=1.0$ before rising back to 219 at $\gamma=50$, indicating that extreme γ forces nearly every training point onto the margin. The polynomial degree validation curve (§5d) confirms that degree $d=1$ achieves the best validation accuracy of 68.9% on Dataset B, with all higher degrees degrading performance as polynomial wiggles memorise the 470 noise dimensions — a textbook illustration of the bias-variance trade-off where model expressiveness without appropriate regularisation invariably harms out-of-sample performance.

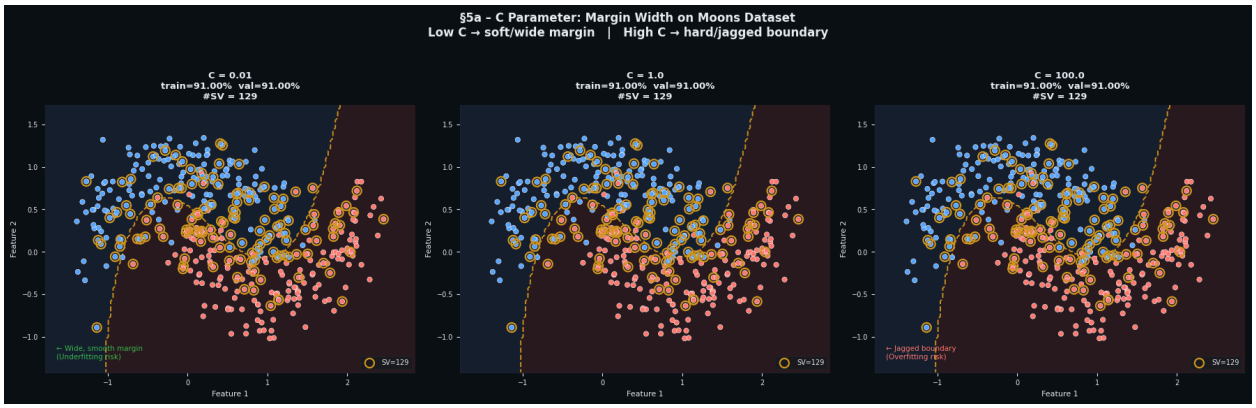
Graphs:

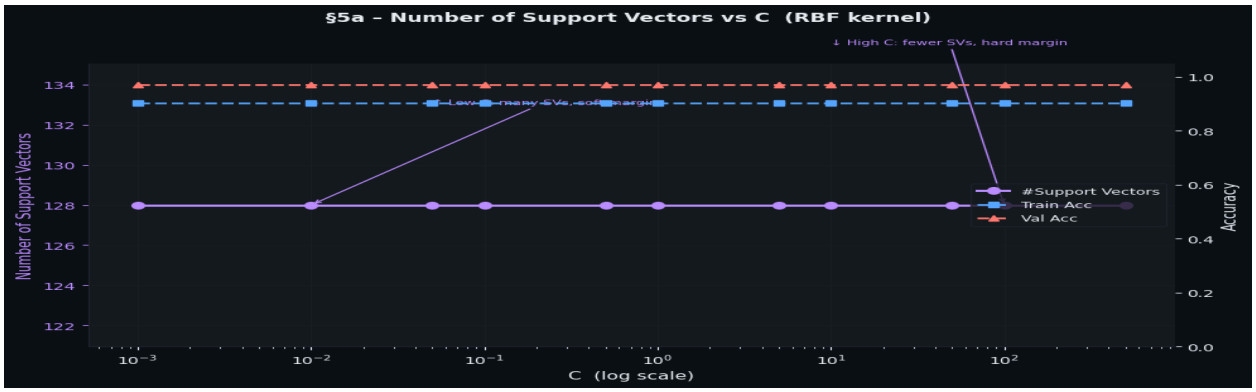

§3a – Experiment 1: Decision Boundaries on Moons Dataset
Perceptron
(step activation)
train=75.00% val=82.00%
Linear SVM
(Pegasos)
train=78.00% val=89.00%
RBF SVM
(γ=0.5, C=5)
train=91.00% val=91.00%
SV=110
SV=129
Feature 1
Feature 2

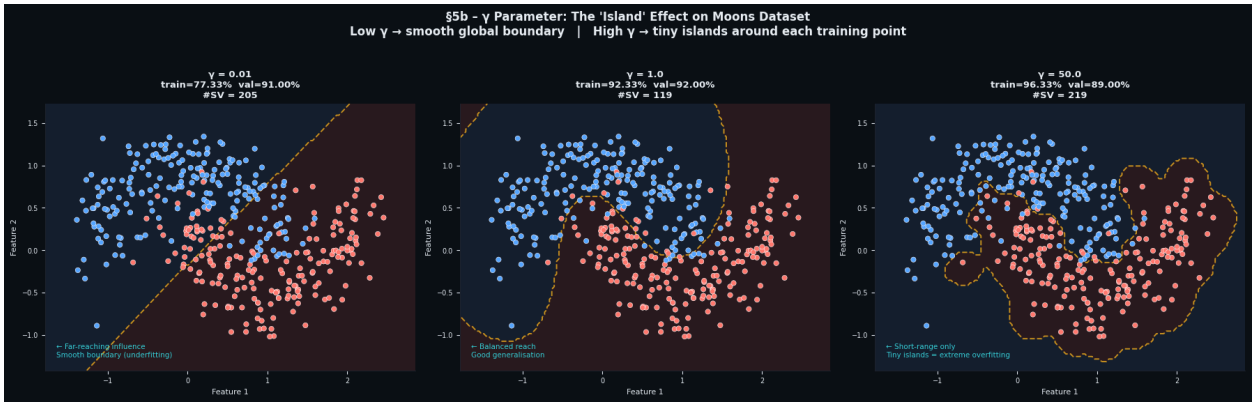

§3b – Experiment 2: NN Overfitting vs SVM Generalisation (Dataset B)
Neural Network (MLP, no regularisation)
NN Train Acc
NN Val Acc
Generalisation Gap
Accuracy
Epoch
Final Accuracy Comparison
100.00%
48.89%
100.00%
51.11%
SVM gap
51.11%
NN gap
48.89%
SVM Train
SVM Val
NN Train (final)
NN Val (final)

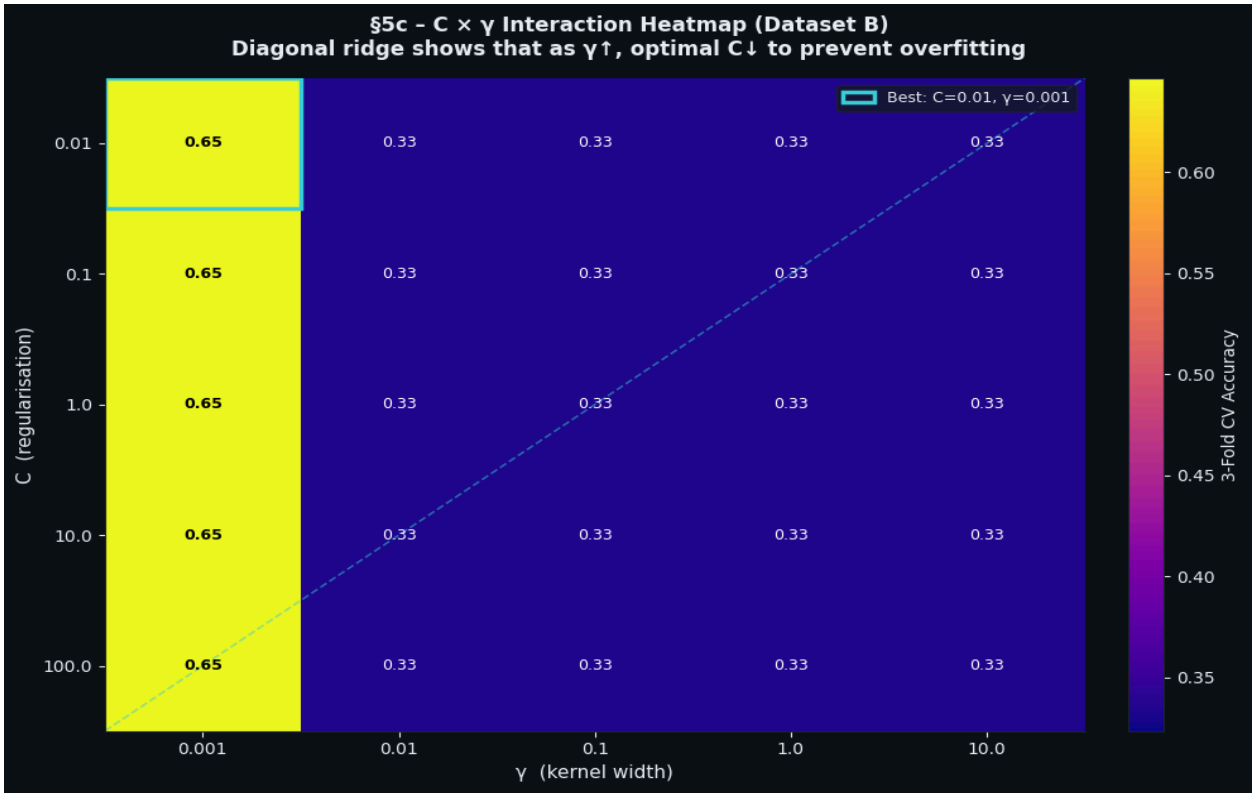

§4 – Kernel Comparison: Decision Boundaries on Dataset A (Moons)
Linear
train=78.00% val=89.00%
Poly (d=3)
train=82.33% val=92.00%
RBF
train=91.00% val=91.00%
Feature 1
Feature 2

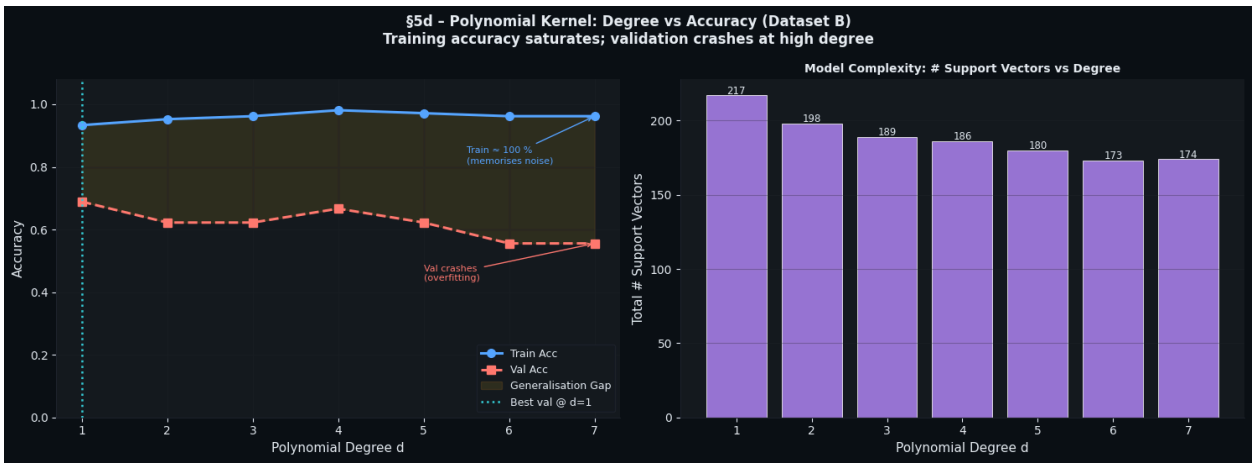

§4 – Kernel Comparison: Train vs Val Accuracy
Dataset A – Moons
Dataset B – High-Dim
Train
Val
#SV (B)
Accuracy
Support Vectors
0.78
0.89
0.82
0.92
0.91
0.91
0.96
0.67
0.98
0.64
1.00
0.49
Linear
Poly (d=3)
RBF


§5a – C Parameter: Margin Width on Moons Dataset
Low C → soft/wide margin | High C → hard/jagged boundary
C = 0.01
train=91.00% val=91.00%
#SV = 129
C = 1.0
train=91.00% val=91.00%
#SV = 129
C = 100.0
train=91.00% val=91.00%
#SV = 129
Feature 1
Feature 2
SV=129


§5a – Number of Support Vectors vs C (RBF kernel)
↓ High C: fewer SVs, hard margin
Low C: many SVs, soft margin
#Support Vectors
Train Acc
Val Acc
Number of Support Vectors
Accuracy
C (log scale)


§5b - γ Parameter: The 'Island' Effect on Moons Dataset
Low γ → smooth global boundary | High γ → tiny islands around each training point
γ = 0.01
train=77.33% val=91.00%
#SV = 205
← Far-reaching influence
Smooth boundary (underfitting)
γ = 1.0
train=92.33% val=92.00%
#SV = 119
← Balanced reach
Good generalisation
γ = 50.0
train=96.33% val=89.00%
#SV = 219
← Short-range only
Tiny islands = extreme overfitting
Feature 1
Feature 2


§5c – C × γ Interaction Heatmap (Dataset B)
Diagonal ridge shows that as γ↑, optimal C↓ to prevent overfitting
Best: C=0.01, γ=0.001
C (regularisation)
γ (kernel width)
3-Fold CV Accuracy
0.01
0.1
1.0
10.0
100.0
0.001
0.01
0.1
1.0
10.0
0.65
0.33
0.60
0.55
0.50
0.45
0.40
0.35


§5d – Polynomial Kernel: Degree vs Accuracy (Dataset B)
Training accuracy saturates; validation crashes at high degree
Accuracy
Polynomial Degree d
Train ≈ 100 %
(memorises noise)
Val crashes
(overfitting)
Train Acc
Val Acc
Generalisation Gap
Best val @ d=1
Model Complexity: # Support Vectors vs Degree
Total # Support Vectors
217
198
189
186
180
173
174